Name : Mohit Gabra

Roll No. 2401027

Batch : T12

Subject : ASD&D

EXPERIMENT NO. 09

AIM: To implement Infrastructure as Code (IaC) using Terraform for provisioning and managing AWS cloud infrastructure.

LO MAPPED : LO4,LO5,LO6

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

LO5: Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

LO6: Develop and demonstrate an end-to-end DevOps pipeline mini-project integrating all stages of DevOps from code commit to deployment and monitoring.

THEORY

Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp. It is used to define, provision, configure, and manage cloud infrastructure using configuration files instead of creating resources manually through a cloud provider's web console.

Terraform supports cloud platforms such as AWS and allows infrastructure resources such as EC2 instances, networks, storage, and other services to be managed through code. This makes infrastructure deployment more consistent, repeatable, automated, and efficient.

Terraform configurations are generally written using HashiCorp Configuration Language (HCL). The configuration file defines the required provider, cloud region, resources, resource properties, and other required settings. Terraform then compares the desired configuration with the existing infrastructure and determines what changes are required.

In this experiment, AWS is used as the cloud provider and an EC2 instance is provisioned using Terraform. The AWS provider allows Terraform to communicate with AWS and perform the required infrastructure operations.

The important Terraform commands used in this experiment are:

- **terraform init** – Initializes the Terraform working directory and downloads the required provider plugins.
- **terraform validate** – Checks whether the Terraform configuration is syntactically correct and valid.
- **terraform fmt** – Formats the Terraform configuration into the standard Terraform style.
- **terraform plan** – Creates an execution plan and shows the resources that Terraform will create, modify, or destroy.

- **terraform apply** – Applies the configuration and provisions the required AWS resources.
- **terraform output** – Displays information about resources after they have been created.
- **terraform destroy** – Removes the infrastructure that was created and managed by Terraform.

Terraform also maintains a state file, generally named terraform.tfstate. The state file contains information about the resources managed by Terraform and helps Terraform determine the current state of the infrastructure.

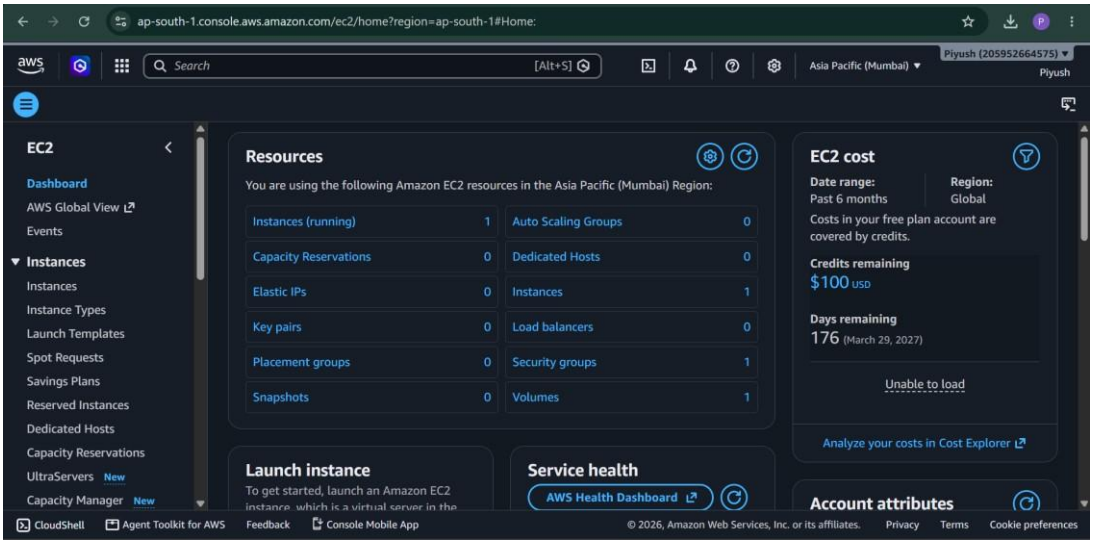
The main advantages of using Terraform are:

- Automation
- Consistency
- Repeatability
- Version Control
- Scalability
- Efficiency

Thus, Terraform provides a systematic approach to managing AWS infrastructure and demonstrates the practical use of Infrastructure as Code in cloud-based engineering.

PROCEDURE STEP 1: Verify AWS Account

Open the AWS Management Console and sign in using the AWS account. Select the Mumbai region (ap-south-1). Open the EC2 Dashboard and ensure that the account has permission to create and manage EC2 resources. For command-line access, an IAM user (Piyush-CLI) was used, as shown in the IAM console below.



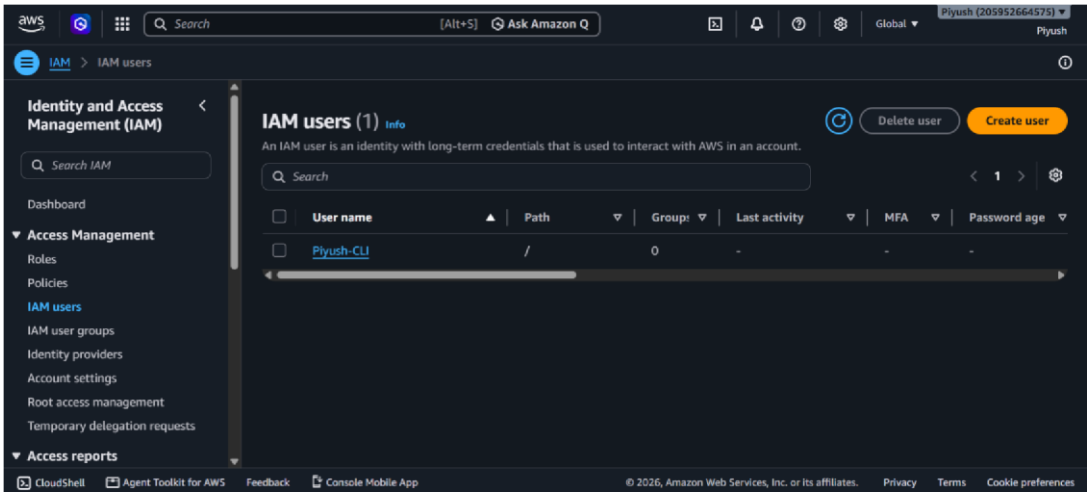


Figure 1: AWS Management Console – IAM Users (IAM User Used for CLI Access)

STEP 2: Configure AWS CLI

Open Windows PowerShell and verify that AWS CLI is installed:

```
aws --version
```

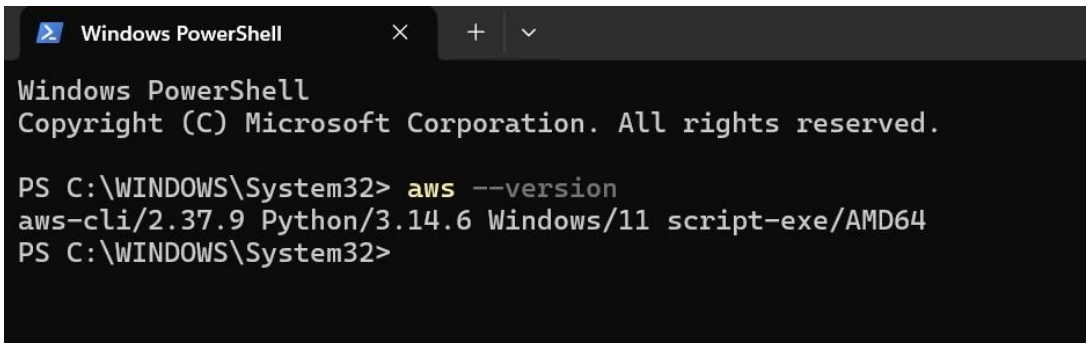


Figure 2: AWS CLI Installation and Version Verification

Configure AWS CLI using the following command and enter the AWS Access Key ID, Secret Access Key, region and output format when prompted. Use the Mumbai region ap-south-1.

```
aws configure
```

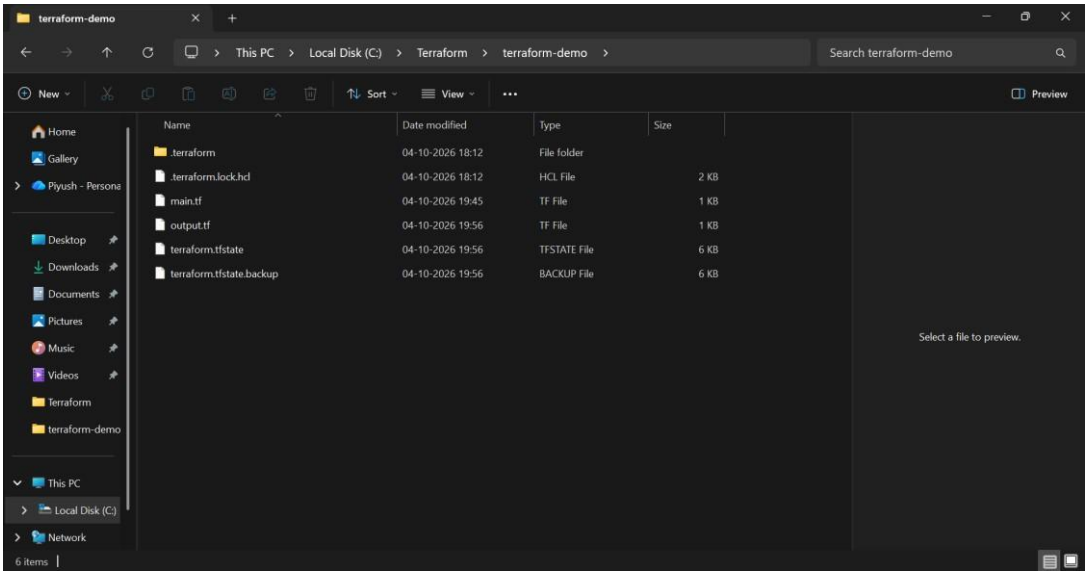
Verify the AWS identity using:

```
aws sts get-caller-identity
```

STEP 3: Install Terraform

Download Terraform for Windows AMD64 from the official Terraform distribution. Extract Terraform into C:\Terraform and add C:\Terraform to the Windows PATH environment variable. Open a new PowerShell window and verify the installation:

```
terraform -version
```



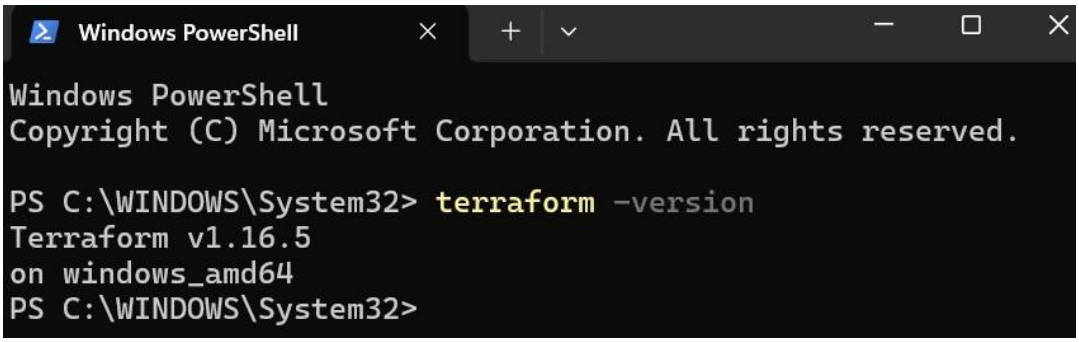


Figure 3: Terraform Installation and Version Verification STEP 4: Create Terraform Working Directory

Create a dedicated folder for the experiment, for example C:\Terraform\terraform-demo, and open PowerShell in this folder. Verify the current directory before creating the Terraform configuration:

```
cd C:\Terraform\terraform-demo
pwd
```

STEP 5: Create main.tf

Create a file named main.tf in the working directory with the following configuration. The instance type was chosen from the free-tier eligible instance types available in the ap-south-1 region, which can be listed using the AWS CLI as shown below.

```
aws ec2 describe-instance-types --filters "Name=free-tier-eligible,Values=true"  --query "InstanceTypes[*].InstanceType"  --output table --region ap-south-1
```

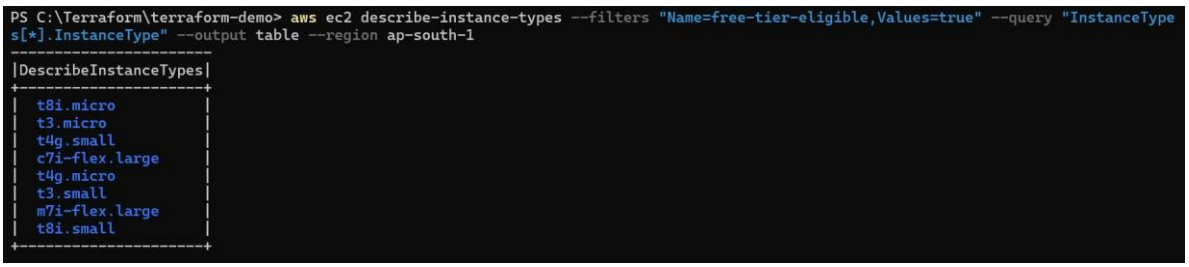


Figure 4: Free-Tier Eligible EC2 Instance Types in ap-south-1

main.tf:

```
terraform {
required_providers {
  aws = {
    source = "hashicorp/aws"
    version = "~> 6.0"
  }
 } }
provider "aws" {
region = "ap-south-1"
} resource "aws_instance" "demo" {
ami = "ami-00412f01c749d8a5f"
instance_type = "t3.micro"
  tags = {
   Name = "Terraform-Demo"
 }
}
```

Explanation:

- **provider "aws"** connects Terraform with AWS.
- **ap-south-1** represents the Mumbai region.
- **aws_instance** creates an EC2 instance.
- **ami** specifies the Amazon Machine Image.
- **instance_type** specifies the EC2 instance type.
- **tags** assigns a name to the instance.

STEP 6: Initialize Terraform

Run the following command. It initializes the Terraform working directory and downloads the required AWS provider plugin (hashicorp/aws), also creating the dependency lock file .terraform.lock.hcl.

```
terraform init
```

```
PS C:\Terraform\terraform-demo> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.67.0...
- Installed hashicorp/aws v6.67.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Terraform\terraform-demo>
```

Figure 5: terraform init Output

STEP 7: Format and Validate the Configuration

Run terraform fmt, which formats the configuration into the standard Terraform style, and then terraform validate, which checks whether the configuration is syntactically correct and valid.

```
terraform fmt
terraform validate
```

```
PS C:\Terraform\terraform-demo> terraform fmt
PS C:\Terraform\terraform-demo> terraform validate
Success! The configuration is valid.

PS C:\Terraform\terraform-demo> terraform plan

Terraform used the selected providers to generate the following
execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.demo will be created
  + resource "aws_instance" "demo" {
      + ami                                  = "ami-00412f01c749d8a5f"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
```

Figure 6: terraform fmt and terraform validate Output (followed by the start of terraform plan) STEP 8: Create Terraform Execution Plan

Run the following command. Terraform compares the desired infrastructure configuration with the current infrastructure and displays the resources that will be created, modified, or destroyed. Here, the plan shows one resource to add, with none to change or destroy.

```
terraform plan
```

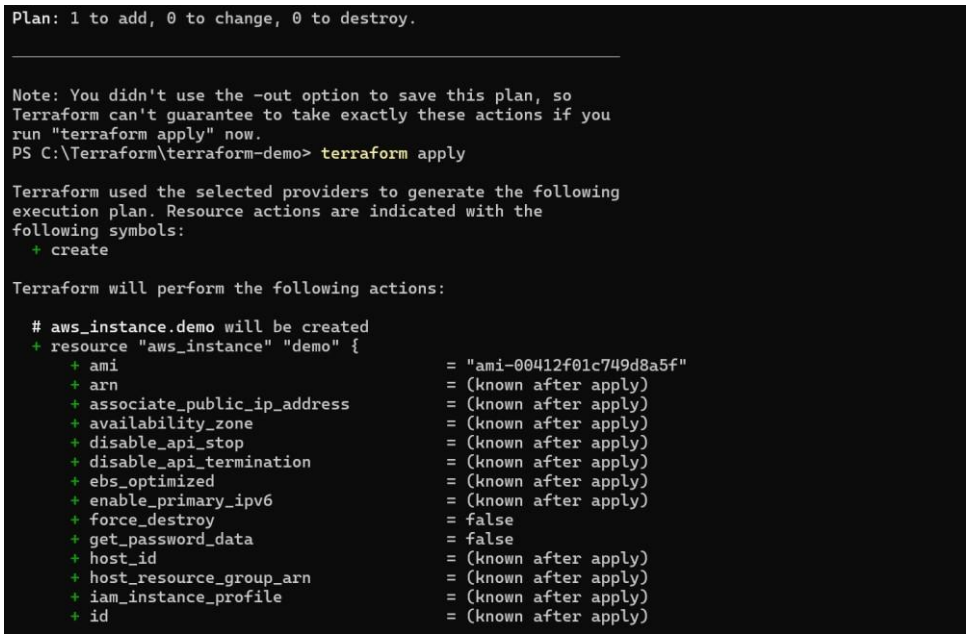


Figure 7: terraform plan Summary (1 to add, 0 to change, 0 to destroy)

STEP 9: Apply Terraform Configuration

Run the following command and, when prompted, enter yes. Terraform provisions the EC2 instance according to the configuration. The output shows successful resource creation with the message Apply complete!

```
terraform apply
```

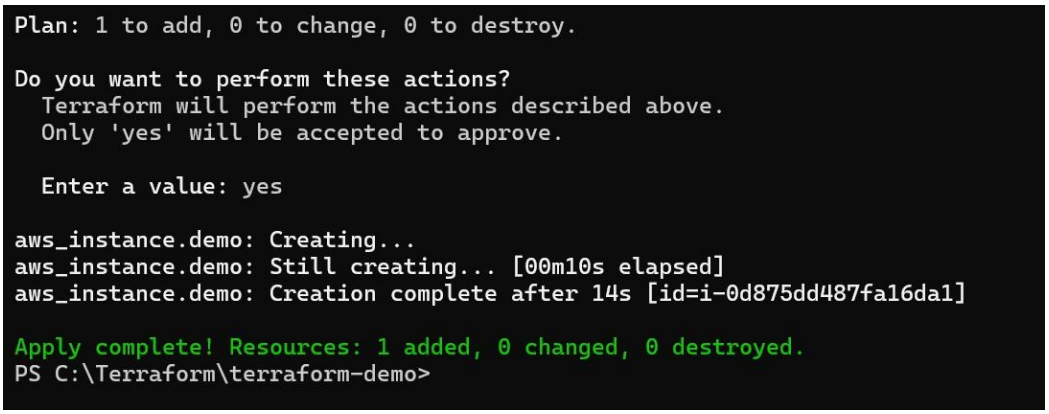


Figure 8: terraform apply Output STEP 10: Verify EC2 Instance in AWS

Open AWS Console → EC2 → Instances and verify that the EC2 instance created by Terraform is visible. Check the instance name/tag (Terraform-Demo), instance state (Running), instance type (t3.micro), and instance ID.

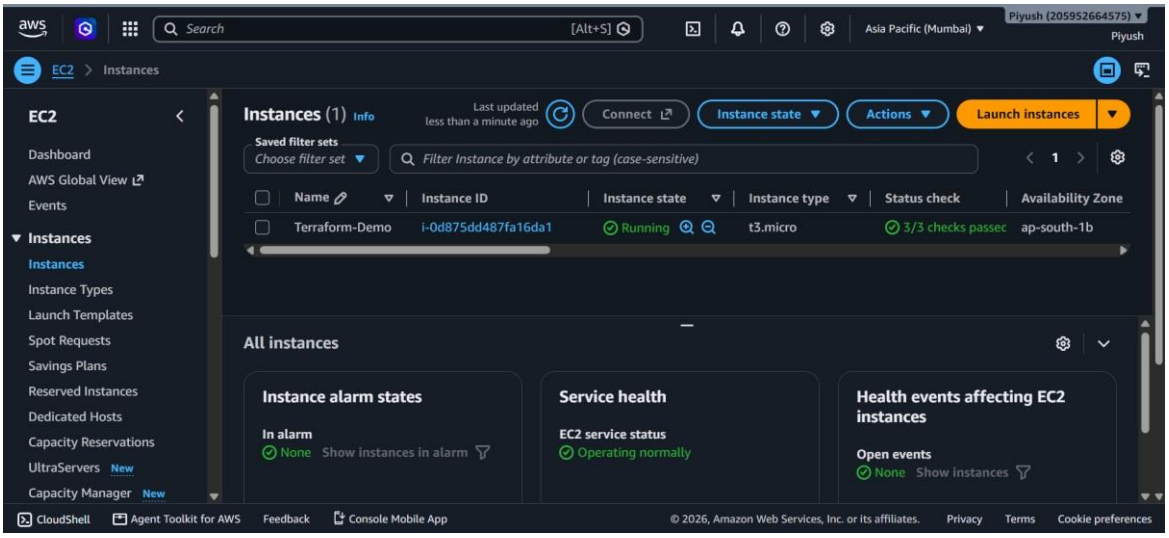


Figure 9: EC2 Instance Provisioned Using Terraform

STEP 11: Create output file

Create a file in the working directory (named output.tf in this experiment) to display useful information about the provisioned EC2 instance:

output "instance_id" { description =

"ID of the EC2 instance" value =

aws_instance.demo.id

}

output "public_ip" { description = "Public IP

address, if assigned" value =

aws_instance.demo.public_ip

}

output "instance_type" { value =

aws_instance.demo.instance_type

}Run terraform apply again so that the output values are saved in the Terraform state. No infrastructure is changed by this run.

```
terraform apply
```

STEP 12: Display Terraform Outputs

Run the following command to display the values defined in the output file, such as the EC2 instance ID, public IP address, and instance type.

```
terraform output
```

```
PS C:\Terraform\terraform-demo> terraform apply
aws_instance.demo: Refreshing state... [id=i-0d875dd487fa16da1]

Changes to Outputs:
  + instance_id   = "i-0d875dd487fa16da1"
  + instance_type = "t3.micro"
  + public_ip     = "65.1.106.155"

You can apply this plan to save these new output values to the
Terraform state, without changing any real infrastructure.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes


Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-0d875dd487fa16da1"
instance_type = "t3.micro"
public_ip = "65.1.106.155"
PS C:\Terraform\terraform-demo> terraform output
instance_id = "i-0d875dd487fa16da1"
instance_type = "t3.micro"
public_ip = "65.1.106.155"
PS C:\Terraform\terraform-demo>
```

Figure 10: terraform apply (Outputs Saved) and terraform output Result

Individual output values can also be displayed by naming them:

```
terraform output instance_id
terraform output public_ip
terraform output instance_type
```

```
PS C:\Terraform\terraform-demo> terraform output instance_id
"i-0d875dd487fa16da1"
PS C:\Terraform\terraform-demo> terraform output public_ip
"65.1.106.155"
PS C:\Terraform\terraform-demo> terraform output instance_type
"t3.micro"
PS C:\Terraform\terraform-demo>
```

Figure 11: Individual Terraform Output Values STEP 13: Verify Terraform Files and State

Verify that the Terraform working directory contains the required files: main.tf, the output file, terraform.tfstate and the .terraform folder. The terraform.tfstate file stores information about the infrastructure managed by Terraform, and terraform.tfstate.backup holds the previous state.

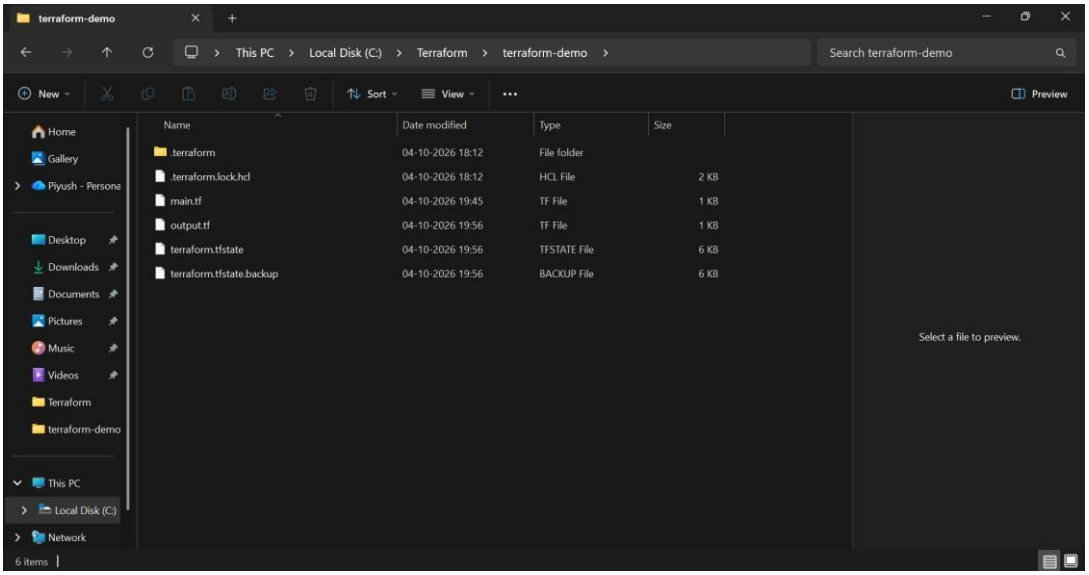


Figure 12: Terraform Project Files and State STEP 14: Destroy the Infrastructure

After completing the verification, remove the resources created by Terraform using the following command and enter yes when prompted. terraform destroy removes the infrastructure managed by the current Terraform configuration.

```
terraform destroy
```

STEP 15: Verify Resource Termination

Return to AWS Console → EC2 → Instances and verify that the Terraform-created EC2 instance has been terminated or removed from the active resources.

LEARNING OUTCOMES ACHIEVED

Students gained practical knowledge of using Terraform to define, provision, and manage AWS cloud infrastructure through Infrastructure as Code (IaC). The experiment helped in understanding:

- Terraform configuration files
- AWS provider configuration
- EC2 provisioning
- Terraform initialization and validation
- Terraform execution plans
- Infrastructure provisioning using terraform apply
- Terraform outputs and state
- Infrastructure removal using terraform destroy

CONCLUSION

The experiment successfully demonstrated the use of Terraform for Infrastructure as Code on AWS. Terraform was used to define, provision, verify, and manage an EC2 instance through configuration files and Terraform commands. The experiment provided practical understanding of automated and repeatable cloud infrastructure management.